

Understanding Array of Pointers in C++

Step 1: Normal 2D Array of Characters

Example: `char names[3][20] = {"Khalid", "Ashraf", "Danish"};` This means 3 rows (names) and 20 columns (characters per name). Each row reserves 20 bytes, even if the name is shorter.

Row	Characters Stored	Memory Used
0	"Khalid"	20 bytes
1	"Ashraf"	20 bytes
2	"Danish"	20 bytes
	Total	60 bytes

Step 2: Array of Pointers

Example: `char *names[] = {"Khalid", "Ashraf", "Danish"};` Here, each element is a pointer to a string, not a full 20-byte block. Each pointer stores the address of the string.

Step 3: Memory Breakdown

Each pointer takes 8 bytes (on a 64-bit system). So: - 3 pointers $\times$ 8 bytes = 24 bytes - Actual strings: "Khalid"(7), "Ashraf"(7), "Danish"(7) = 21 bytes Total = 24 + 21 = 45 bytes.

Step 4: Why It Saves Memory

If strings have different lengths, array of pointers is more efficient because it only uses the exact memory needed for each string, plus the pointers.

Method	How It Stores	Total Memory (Example)	Efficiency
<code>char names[3][20]</code>	Fixed 20 bytes per name	60 bytes	■ Wastes space
<code>char *names[]</code>	Pointers + actual strings	≈45 bytes	■ Saves space

In Short:

Array of pointers uses less memory because it stores addresses, not fixed-length blocks. It's ideal when strings have different lengths.